Backpropagation — Exam Question 1

1.1 A scalar sigmoid unit. Consider the scalar model $f = \sigma(Wx + b)$.

(a) Decompose the forward pass into its primitive operations. (2 pts)

(b) Using the variables from (a), draw the computation graph. (2 pts)

(c) For $x = 1$, $W = 2$, $b = 3$, target $y = 1$, and squared-error loss $L = (f - y)^2$, compute the forward pass and the gradient at each node of the graph. You may annotate the graph or write forward and backward values for each operation. (6 pts)

(d) Derive $\partial f / \partial W$ analytically. You may use $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. (1 pt)

1.2 The sigmoid-derivative curve. Fig. 1 plots the $\partial f / \partial W$ formula you derived in 1.1(d), evaluated at $x = 1$, $b = 0$, as a function of W.

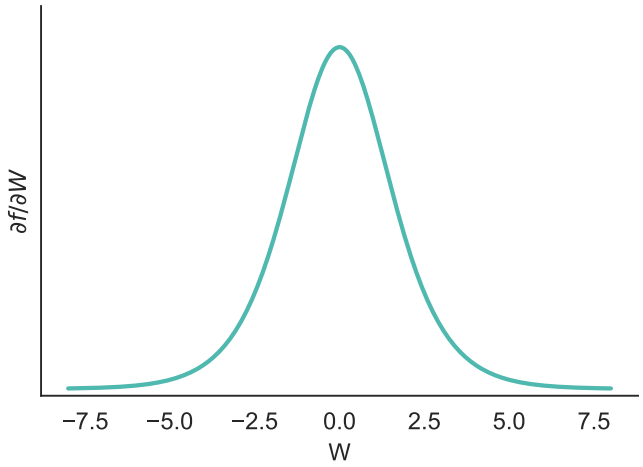


Fig. 1

(a) Explain the shape of the curve in Fig. 1. (1 pt)

(b) Using the chain rule, derive $\partial L / \partial W$ for the scalar model under $L = (f - y)^2$. (1 pt)

(c) Explain why $\partial L / \partial W = 0$ when $f = y$. (1 pt)

(d) State the gradient descent update rule for W. Why would large initial values of W lead to poor loss minimization? (2 pts)

1.3 Two-layer scalar composition. Now consider the two-layer composition

$$G = \sigma(W_2 \sigma(W_1 x + b_1) + b_2)$$

and the same squared-error loss $L = (G - y)^2$.

(a) Draw the computation graph for the full model and loss. (1 pt)

(b) Compute the forward pass and the gradient at each node, for scalar values $x = 1$, $W_1 = 2$, $b_1 = 1$, $W_2 = 4$, $b_2 = 1$, target $y = 1$. (4 pts)

(c) Explain why the gradient of W_1 is much smaller than the gradient of W_2. (1 pt)

1.4 Fully-connected variant and weight initialization. Consider a fully-connected variant of the two-layer model from 1.3:

$$h = \sigma(\boldsymbol{W}_1 x + b_1), \quad G = \sigma(\boldsymbol{W}_2 h + b_2),$$

where the input $x \in \mathbb{R}^2$ is now a 2-feature vector, $\boldsymbol{W}_1 \in \mathbb{R}^{4\times 2}$ and $\boldsymbol{W}_2 \in \mathbb{R}^{1\times 4}$ are weight matrices (boldface to distinguish them from the scalar W_1, W_2 of 1.3), $b_1 \in \mathbb{R}^4$, $b_2 \in \mathbb{R}$, and $h \in \mathbb{R}^4$ is the hidden activation. The scalar chain from 1.3 corresponds to a single *stream* of this network — one entry of $\boldsymbol{W}_1$ times one entry of $\boldsymbol{W}_2$, shown as the highlighted path in Fig. 2 — which reduces exactly to the scalar 1.3 model only when every off-path entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ is zero.

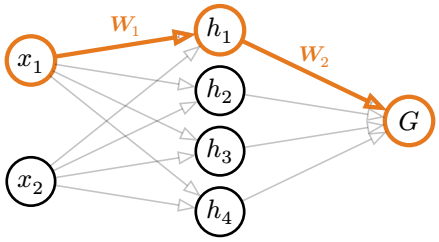


Fig. 2

We train this network on the two-moons binary classification dataset (Fig. 3). The four loss curves (A–D) in Fig. 4 correspond to four different weight initializations ($\eta = 0.01$ for 5000 epochs): A — all zeros; B — Gaussian ($W_{ij} \sim \mathcal{N}(0, 1)$); C — large magnitude ($W_{ij} \sim \mathcal{N}(0, 10^2)$); D — symmetric ($W_{ij} = 1$ for every entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$).

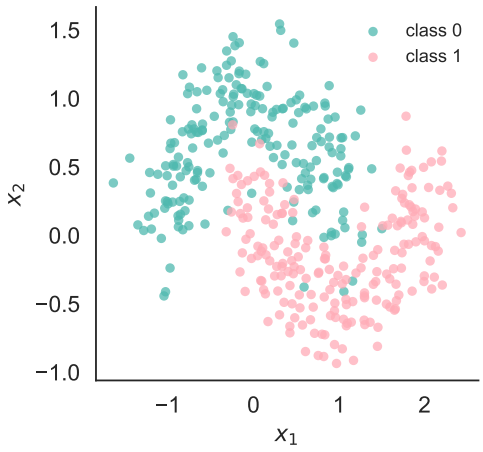


Fig. 3

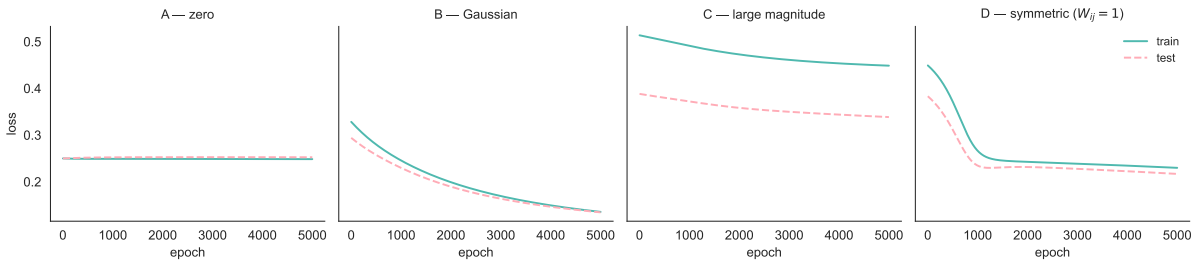


Fig. 4

(a) Decompose the forward pass into its constituent operations, annotating the shape of each intermediate at every step. (2 pts)

(b) In one sentence per plot, explain the behavior of each of the four curves, tying back to your analyses in 1.2 and 1.3. (4 pts)

(c) The dataset is a binary classification task. In addition to the squared-error loss we have used so far, name one other loss that would be appropriate here and briefly justify the choice. (1 pt)

1.5 LayerNorm. Consider an extension of the model in 1.4 where a *LayerNorm* is inserted after the first linear layer and before the hidden sigmoid:

$$z_1 = \boldsymbol{W}_1 x + b_1, \quad \hat{z}_1 = \frac{z_1 - \mu}{s}, \quad h = \sigma(\hat{z}_1),$$

where $h \in \mathbb{R}^4$ is the hidden activation passed to the output layer (same role as in 1.4), and μ, s are the per-sample mean and standard deviation of z_1 across the four hidden units:

$$\mu = \frac{1}{4}\sum_{i=1}^{4} (z_1)_i, \quad s = \sqrt{\frac{1}{4}\sum_{i=1}^{4} \big((z_1)_i - \mu\big)^2}.$$

Fig. 5 shows training loss on the moons data under the same poor (large-magnitude) initialization as 1.4(C), both with and without LN (left panel). The right panel shows the magnitude of the gradient at every entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$, captured at epoch 1000 (dashed grey line in the left panel), when the performance of the non-LN model is only slowly improving.

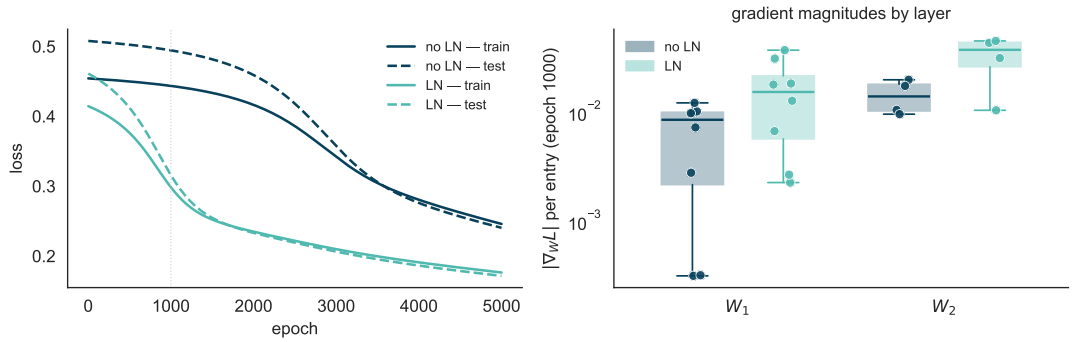


Fig. 5

(a) Redraw the computation graph for a single stream of this modified network. (2 pts)

(b) What is the added term to the chain of gradients flowing through the network? (1 pt)

(c) Using the $\partial f/\partial W$ curve in Fig. 1, explain how LayerNorm shifts the pre-activation z_1 toward the region where σ' is large, and why this makes the LN variant more *robust* to different initialization scales of $\boldsymbol{W}_1$. (3 pts)

(d) Using the right panel of Fig. 5, explain why the LayerNorm variant converges much faster than the no-LN model. (1 pt)

(e) Using the $\partial f/\partial W$ curve in Fig. 1, explain why the gradient magnitude is larger at both $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ under LayerNorm. (1 pt)

1.6 K-nearest neighbors. A *K-nearest-neighbors* (KNN) classifier assigns to each input the majority label among its K closest points in the training set (by Euclidean distance). Unlike the sigmoid MLP from 1.4, KNN has no trainable parameters — it simply stores the training data and computes distances at prediction time.

Fig. 6 shows the decision boundaries of four classifiers on the moons training data: the MLP from 1.4(B) (Gaussian init, trained to convergence) and three KNN classifiers. Which is which has been anonymized as "Model A"–"Model D". The training set contains $N_0 = 106$ class-0 points and $N_1 = 94$ class-1 points ($N = N_0 + N_1 = 200$).

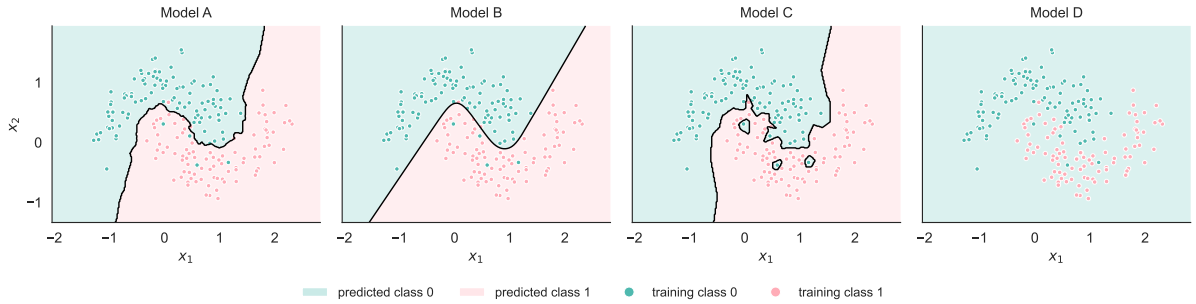


Fig. 6

(a) Name one practical advantage and one practical disadvantage of KNN relative to the trained MLP. (2 pts)

(b) The NumPy skeleton below implements a KNN classifier for binary labels. Fill in the three blanks (marked `...`) so that the function returns the predicted label for each row of `X_test`. (3 pts)

```
import numpy as np

def knn_predict(X_train, y_train, X_test, K):
    """
    X_train: (N, d) training inputs
    y_train: (N,)   training labels in {0, 1}
    X_test:  (M, d) query points
    Returns: (M,) predicted labels in {0, 1}
    """
    preds = np.zeros(X_test.shape[0])
    for i, x in enumerate(X_test):
        # 1. Euclidean distances from x to every training point: shape (N,)
        dists = ...
        # 2. indices of the K nearest training points
        idx   = ...
        # 3. majority label among those K neighbors
        preds[i] = ...
    return preds
```

(c) What is the computational cost of a single prediction at test time, as a function of the training-set size N and the input dimension d? How does this compare with a forward pass through the 2-4-1 MLP from 1.4? (1 pt)

(d) The sigmoid MLP was trained by gradient descent. Why can a KNN classifier *not* be trained the same way? (1 pt)

(e) Three of the four panels (A, B, C) show KNN classifiers with $K \in \{1, 5\}$ together with the MLP. Match each of Models A, B, C to one of {MLP, KNN with $K = 1$, KNN with $K = 5$}, and justify your assignments from the shape of the boundaries. (2 pts)

(f) Panel D shows a KNN classifier with $K = N = 200$. Explain why its prediction is constant across the plane, referring to the class balance stated above. (1 pt)